Integration, Channels & Day 3 Wrap-up

From Script to Service

Day 3 - Session 4

KavinSchool

Session 4: What We'll Cover

1. Integration patterns: wrapping chatbots in APIs
2. Handling common queries: caching and shortcuts
3. Error handling for production
4. Day 3 recap & Day 4 preview

Integration Patterns

Making Your Chatbot Accessible

KavinSchool

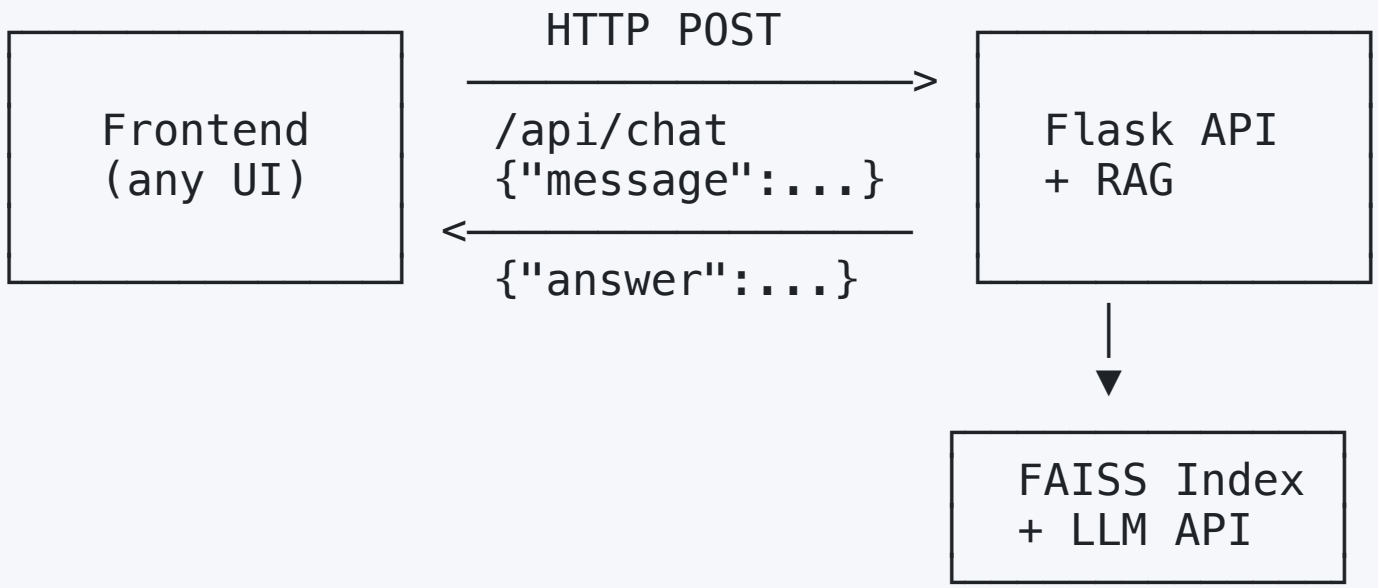
From Script to Service

Your chatbot works in the terminal. How do you make it accessible to users?

Pattern	What It Is	Complexity
CLI script	Terminal-based (what we have)	Low
REST API	HTTP endpoints (Flask/FastAPI)	Medium
Web frontend	React/HTML chat UI	Medium-High
Slack/Teams bot	Chat platform integration	Medium
SDK/Library	Importable Python package	Low-Medium

> Today we build the **REST API** layer. A frontend can call it later.

REST API: The Architecture



The API handles: request parsing, RAG query, response formatting, error handling.

Flask API: Minimal Example

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route("/api/chat", methods=["POST"])

def chat():
    data = request.get_json()
    user_message = data.get("message", "")

    if not user_message:
        return jsonify({"error": "Message is required"}), 400

    # Run RAG query
    result = rag_query(user_message, index, chunks)
    return jsonify({
        "answer": result["answer"],
        "sources": result["sources"],
    })

if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

Testing the API

```
# Using curl
curl -X POST http://localhost:5000/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "What is the vacation policy?"}'

# Response
{
  "answer": "According to the employee handbook...",
  "sources": ["vacation-policy.md"]
}
```

```
# Using Python requests
import requests

response = requests.post("http://localhost:5000/api/chat",
    json={"message": "What is the vacation policy?"})
print(response.json())
```

Adding Conversation Support

```
# Track sessions with a simple dict
sessions = {}

@app.route("/api/chat", methods=["POST"])
def chat():
    data = request.get_json()
    session_id = data.get("session_id", "default")
    user_message = data.get("message", "")

    # Get or create session history
    if session_id not in sessions:
        sessions[session_id] = []
    history = sessions[session_id]

    # RAG query with history context
    result = rag_query(user_message, index, chunks)

    # Update history
    history.append({"role": "user", "content": user_message})
    history.append({"role": "assistant", "content": result["answer"]})

    return jsonify({"answer": result["answer"],
                    "sources": result["sources"],
                    "session_id": session_id})
```

Production Considerations

Caching, Error Handling & Robustness

KavinSchool

FAQ Caching

For **frequently asked questions**, cache the response to avoid repeated LLM calls:

```
from functools import lru_cache
import hashlib
response_cache = {}

def cached_rag_query(query, index, chunks):
    """Cache RAG responses for identical queries."""
    cache_key = hashlib.md5(query.lower().strip().encode()).hexdigest()

    if cache_key in response_cache:
        print(f"Cache hit for: {query[:50]}...")
        return response_cache[cache_key]

    result = rag_query(query, index, chunks)
    response_cache[cache_key] = result
    return result
```

> In production, use Redis or Memcached. In-memory caching is fine for prototypes.

Error Handling

```
import litellm

@app.route("/api/chat", methods=["POST"])
def chat():
    try:
        data = request.get_json()
        if not data or "message" not in data:
            return jsonify({"error": "JSON body with 'message' required"}), 400

        result = rag_query(data["message"], index, chunks)
        return jsonify(result)

    except litellm.exceptions.RateLimitError:
        return jsonify({"error": "Rate limited. Try again shortly."}), 429

    except litellm.exceptions.AuthenticationError:
        return jsonify({"error": "API key error. Contact admin."}), 500

    except Exception as e:
        return jsonify({"error": f"Internal error: {str(e)}"}), 500
```

Deterministic Shortcuts

Skip the LLM for queries you can answer directly:

```
SHORTCUTS = {
    "hello": "Hi! I'm your knowledge base assistant. Ask me anything!",
    "help": "I can answer questions about our documentation. "
            "Try asking about vacation policy, coding standards, etc.",
}

def handle_query(query):
    """Check shortcuts first, then fall back to RAG."""
    normalized = query.lower().strip()

    # Check shortcuts
    for trigger, response in SHORTCUTS.items():
        if normalized == trigger:
            return {"answer": response, "sources": ["shortcut"]}

    # Fall back to RAG
    return rag_query(query, index, chunks)
```

Integration: Slack / Teams (Concept)

The **pattern** for any chat platform:

```
# 1. Receive message from platform webhook
@app.route("/slack/events", methods=["POST"])
def slack_event():
    event = request.json
    user_message = event["event"]["text"]

    # 2. Run your RAG query
    result = rag_query(user_message, index, chunks)

    # 3. Send response back to platform
    send_slack_message(
        channel=event["event"]["channel"],
        text=result["answer"]
    )
    return "", 200
```

> The LLM logic is **identical**. Only the input/output transport changes.